

Causal Consistency as a Defense: Detecting and Auditing Provenance Graph Poisoning

N. Manoj Independent Researcher
Email: nehamanoj1105@users.noreply.github.com

Abstract

Provenance-based intrusion detection consumes a stored graph and assumes it is authentic. We examine what happens when that assumption fails, and whether the causal invariants that a provenance graph must satisfy by construction are sufficient to reveal post-collection tampering without any training. We assemble fifteen deterministic rules over structural, temporal and semantic invariants into a rule engine and evaluate it against a GraphSAGE baseline on controlled synthetic poisoning and on real DARPA Transparent Computing Engagement 3 Theia provenance. Over ten seeds, the rule engine reaches F1 0.805 ± 0.063 on synthetic poisoning, with a genuine continuous score giving ROC-AUC 0.953 ± 0.031 , while a leakage-free GraphSAGE reaches F1 0.322 ± 0.194 ; the paired difference is $+0.482$ (Wilcoxon $p = 0.002$). The rule engine’s accuracy is unchanged as invariant-preserving mimicry grows from none to heavy. On real Theia graphs with synthetic post-collection poisoning it reaches F1 0.835 ± 0.046 and 0.850 ± 0.060 , where GraphSAGE is near chance. We give particularly close attention to evaluation design: the conventional protocol of training and thresholding a baseline on the same edges inflates GraphSAGE F1 by 0.342 absolute, and a reported ROC-AUC of 1.000 for the rule engine is not a computed value. We separate real from synthetic evaluation throughout and release the complete corrected protocol, per-seed results and verification scripts.

Index Terms

provenance, intrusion detection, graph poisoning, causal consistency, GraphSAGE, evaluation methodology, reproducibility, DARPA Transparent Computing

I. INTRODUCTION

The provenance of a computer system is its causal history: which process read which file, which process spawned which child, which process opened which network connection. Captured faithfully, it is a directed graph rich enough to reconstruct an intrusion after the fact and, increasingly, to detect it as it happens. Systems such as Unicorn [1], NoDoze [11], HOLMES [9], Poirot [10], FLASH [6] and MAGIC [5] build on the assumption that the provenance graph is a trustworthy record.

That assumption holds only up to the point where the graph stops being produced and starts being stored. Once events are serialized, shipped and indexed, the graph becomes data, and data can be edited by an adversary with access to the storage. Deleting the events that record a privilege escalation, inserting events that suggest a process behaved benignly, reordering events to break a causal chain, or re-attributing an action to an innocent actor are all plausible operations against a stored graph. A downstream detector then reasons over a fiction with no indication that anything is wrong.

Our question is whether causal consistency is enough to catch this. The events in a provenance graph are not arbitrary; they must satisfy invariants that come from what the events mean. An execute edge must point at a file. A read edge must point at a file. A child process cannot be spawned before its parent exists. Event sequence numbers within a process must increase. A process that appears to act must at some point have been spawned. These invariants are cheap to check and require no training data, which raises the question of how far they take us against an adversary who edits the graph.

We answer with an empirical study and, deliberately, a methodological one. We build a fifteen-rule engine from these invariants and compare it to GraphSAGE, a representative learned detector, on the same poisoned instances. The comparison favors the rule engine, but the more useful result is why: a learned baseline evaluated in the conventional way—fit on all edges, threshold chosen to maximize F1 on the scored labels—has most of its reported advantage supplied by that choice. When the split is made honest, the advantage largely disappears. Because this failure mode is easy to reproduce and easy to miss, we treat it as a first-class result rather than a footnote.

A. Contributions

- A deterministic, training-free detector for post-collection provenance tampering, built from fifteen structural, temporal and semantic invariants (Section VI).
- A rigorous evaluation protocol with disjoint train, validation and test edges, validation-only threshold selection, and test-set isolation from weights, threshold, hyperparameters and attack intensity (Section VIII).
- A ten-seed comparison on synthetic poisoning and an evaluation on two real Theia scenarios with synthetic post-collection poisoning, with paired significance tests (Sections XI–XIII).
- A genuine continuous anomaly score for the rule engine, replacing a reported ROC-AUC of 1.000 that was a fallback constant rather than a computation; the computed value is 0.953 ± 0.031 (Section XVII).
- A leakage analysis quantifying a 0.342 absolute F1 inflation for the conventional GraphSAGE protocol (Section XVI).

TABLE I
THREAT MODEL SUMMARY.

Aspect	Assumption
Trusted	kernel, audit logging, parser, detector
Untrusted	stored graph, intermediate representations, storage
Attacker can	delete, insert, reorder, forge edges; edit attributes
Attacker cannot	modify detector/parser, compromise kernel
Knowledge	schema + detector class (gray-box)
Goal	hide malicious activity, preserve structure

- A complete release of protocols, raw per-seed results and verification scripts so that every number can be re-derived (Section XXXII).

II. BACKGROUND AND RELATED WORK

A. Provenance-based detection

Specification-based systems (SLEUTH [8], HOLMES [9], Poirot [10]) encode expert knowledge of attack behavior and are precise but require continued maintenance and do not adapt to novel tactics. Statistical systems score the rarity of events or paths (NoDoze [11], ProvDetector), and are prone to flagging unusual-but-benign activity. Learning-based systems learn representations of benign behavior: Unicorn [1] at coarse granularity, FLASH [6] and MAGIC [5] at finer granularity, with GNN backbones such as GraphSAGE [12] and GAT.

B. Adversarial robustness of graph detectors

Graph-based detectors have been shown vulnerable to mimicry, in which an adversary interleaves benign activity with malicious activity so that local aggregation dilutes the anomaly signal [3]. Defenses have moved toward dynamic neighbor selection and reinforcement-learned aggregation. Our setting differs: the adversary is not trying to look benign *inside* the graph, but to edit the graph itself. A detector that verifies invariants is therefore addressing a different question than a detector that classifies behavior.

C. Provenance integrity

The integrity problem for provenance has mostly been approached through capture-side mechanisms—trusted logging, tamper-evident storage, cryptographic chaining. Our work is complementary: given a graph whose custody is uncertain, we ask how much can be recovered from internal consistency alone, as a lightweight layer before any cryptographic mechanism is available.

III. PROBLEM FORMULATION

Let $G = (V, E)$ be a provenance graph produced by a capture pipeline and then stored. We consider an adversary that modifies E between capture and analysis. The analysis receives a graph G' and must decide, for each edge, whether it witnesses tampering. Formally, we seek a detector $D : G' \rightarrow \{0, 1\}^{|E'|}$ and, ideally, a score $s : G' \rightarrow \mathbb{R}^{|E'|}$.

The ground truth is the set of edges the adversary changed. This is well defined for insertion, reordering and forgery, because those operations leave a (modified) edge in G' . It is not well defined for deletion, because the deleted edge is, by construction, absent from G' ; an edge-level detector cannot mark an edge that is not there. We therefore do not treat deleted edges as scored positives. Their detectability comes only from breaking a higher-level invariant elsewhere—a parent process left without a spawn record, a gap in a sequence—and we report deletion separately.

IV. THREAT MODEL

We assume the attacker can read and write the stored graph, but cannot modify the detector, disable the parser, or compromise the kernel that produced the records (Table I). The attacker may delete, insert, reorder and forge edges, and may edit node and edge attributes. The attacker is gray-box: the schema and the general class of detector are known, but not detector parameters or learned weights. The goal is to hide malicious activity while preserving as much observable structure as possible. Timestamp manipulation, attribute forgery and targeted stealth poisoning are in scope as future attack classes but are not evaluated here.

V. PROVENANCE REPRESENTATION

A provenance graph $G = (V, E)$ has typed nodes and edges. Node types are process, file, user and network endpoint. Edge types are read, write, execute, connect, spawn and delete. Each edge carries source and target identifiers, a type and a timestamp; each event carries a sequence number within its owning process. This is the common data model of the DARPA Transparent Computing program, whose records are serialized in Avro; our parser decodes that schema into the representation above. The rules below are stated entirely in terms of these types, the identifiers and the timestamps, so they apply to any capture that produces this model.

VI. PROPOSED DETECTION FRAMEWORK

The detector is a rule engine that evaluates fifteen invariants over the graph and returns, for each, the set of violations, each attributed to an edge or a node. We group the rules by the kind of invariant they check.

A. Structural rules

Well-formedness of types and topology: no duplicate edge tuples; no duplicate event identifiers; spawn edges must target process nodes; execute, read, write and delete edges must target file nodes; connect edges must target network endpoints; no self-referential edges; no edges referencing nodes that are not present; and no active process lacking a parent spawn edge.

B. Temporal rules

Ordering invariants: non-negative timestamps; a child's spawn must not precede its parent's spawn; process activity must not precede the process spawn; event sequence numbers must be contiguous enough to reveal gaps; and timestamps must be monotone within a process stream.

C. Semantic rules

Lineage invariants: a process that acts without any observed parent spawn is flagged, capturing attempted laundering of provenance through re-attribution.

D. Continuous anomaly score

For ranking and for AUC, every violation contributes a severity weight (high 3, medium 2, low 1) to the edge it is attributed to, either directly by edge identifier or, for node-attributed violations, to the incident edges. The binary decision is score greater than zero. This score is a genuine, monotone ranking signal derived from the violation structure; we use it, and only it, when reporting ROC-AUC and PR-AUC. No AUC in this paper is produced by a fallback constant.

VII. GRAPH SAGE BASELINE

The learned baseline is a two-layer GraphSAGE encoder [12] with an edge classifier. Nodes are featurized by a one-hot node type and degree; edges by a one-hot edge type and a normalized timestamp. The baseline exists to test whether learning can exceed fixed invariants under a fair protocol. It is not tuned to win, its feature set is deliberately minimal, and Section XVI shows that its apparent strength in the original repository came largely from the evaluation protocol rather than from the model.

VIII. EXPERIMENTAL METHODOLOGY

A. Datasets

Two data sources are used. The first is a synthetic provenance generator producing a graph of thirty processes, forty files and ten network entities, which provides exact ground truth and allows the attack intensity to be varied at will. The second is real provenance from DARPA Transparent Computing Engagement 3, the Theia streams [13]. Four benchmark scenarios are named in the original material (1r, 3, 5m, 6r). Two were obtained and parsed: scenario 3 with 20,157 nodes and 297,777 edges, and scenario 5m with 34,835 nodes and 464,858 edges. Both parse counts match the documented statistics exactly. Scenarios 1r and 6r could not be downloaded, because the public mirror enforced a download quota (HTTP 403); we report the two obtained scenarios and state the missing coverage as a limitation rather than estimating it (Table XIV).

Real-data experiments are run on the first 50,000 edges of each real graph, matching the loader's behavior; the truncation is explicit. We emphasize the category separation used throughout: the real-data experiments are *real provenance with synthetic post-collection poisoning*. They are not real attack labels, and we never describe them as detection of naturally occurring attacks.

B. Poisoning operators

For each seed we inject five deletions, five insertions, five reorderings and five dependency forgeries, for twenty events. The injector operates on a deep copy of the clean graph, records for each event its original state, its modified state, an expected detection target and a flag indicating whether the operation actually changed the graph. It then asserts that the clean graph is unmodified. This matters because a naive implementation can mutate its input in place, and because some reorderings can be no-ops if they do not change the semantic order; we mark such events ineffective and exclude them from detectable positives.

Twenty events yield fifteen scored positives, since the five deletions leave no final-graph edge. Detection metrics are computed over the final graph (*final-graph universe*). An alternative convention, which adds the identifiers of deleted edges to the scoring universe, raises F1 from 0.805 to 0.845; this is quantified in Section XIV and we use the final-graph universe in all reported results.

C. Leakage-free protocol

For the learned baseline, edges are partitioned into disjoint training, validation and test sets. Weights are fit on the training edges only; the decision threshold is selected on the validation edges only; and every reported metric is computed on the untouched test edges. Message passing for test-node features uses training and validation edges only, so test edges influence neither model weights nor threshold. Neither hyperparameters nor attack intensity is tuned on the test set. For the training-free rule engine there is no split to leak, which is precisely why the leakage below affects only the baseline.

D. Metrics

We report precision, recall, F1, accuracy, balanced accuracy, MCC, FPR, FNR, specificity, ROC-AUC and PR-AUC. ROC-AUC and PR-AUC are reported only where a genuine continuous score exists and both classes are present. For the rule engine this is the severity-weighted score of Section VI; for GraphSAGE it is the classifier probability.

E. Seeds and statistics

Synthetic, mimicry, ablation and real-Theia experiments use ten seeds (1, 7, 13, 21, 42, 99, 123, 256, 512, 1024); generalization uses five (1, 7, 13, 21, 42). Each seed produces a freshly generated graph and a fresh set of attack instances. We report the mean, sample standard deviation, median, minimum, maximum and a 95% confidence interval (Student- t , $n - 1$ degrees of freedom). Paired comparisons between detectors use the Wilcoxon signed-rank test on the same test instances, with Cohen’s d_z as an effect size.

IX. DATA PARSING AND PREPROCESSING

Raw Theia records are Avro-serialized common data model entries nested under a datum envelope. The parser walks each record, extracts the typed subject and predicate, and emits an edge with source, target, type and timestamp. Nodes are materialized on demand from identifiers. Parsing the two obtained scenarios yields zero skipped records, and the counts agree with the published statistics, which is our validation that the decode is correct. No attack labels are read from the real data; the DARPA ground-truth report was obtained for context only and is not used as a detection target.

X. EVALUATION PROTOCOL

Detection is evaluated at the edge level against the injected ground truth, under the final-graph universe. For the rule engine, the binary decision is score greater than zero and the continuous score is used for AUC. For GraphSAGE the protocol is the leakage-free one of Section VIII-C. Real and synthetic evaluations are never pooled: Table IV is synthetic, Table VI is real-provenance-with-synthetic-poisoning, and the two are reported and discussed separately.

XI. MAIN DETECTION RESULTS

Table IV reports the ten-seed synthetic comparison. The rule engine reaches F1 0.805 ± 0.063 , with precision 0.715 ± 0.095 , recall 0.933 ± 0.063 and specificity 0.964 ± 0.017 ; MCC is 0.795 ± 0.064 . The leakage-free GraphSAGE reaches F1 0.322 ± 0.194 , precision 0.342 ± 0.309 and recall 0.502 ± 0.364 . The learned baseline is not competitive once evaluated honestly, and its seed-to-seed variance is roughly three times larger. Under a paired test over the same instances, the F1 difference is $+0.482$ with a 95% confidence interval of $[0.328, 0.636]$, Cohen’s $d_z = 2.24$ and a Wilcoxon $p = 0.002$; the ROC-AUC difference ($+0.124$, $p = 0.105$) is not statistically distinguished at this sample size (Table XIII). We report that non-result rather than claiming a uniform advantage.

Figure 1 shows the per-metric comparison. The rule engine’s high recall and specificity are the expected signature of a detector that checks invariants: it catches almost all injected structure while rarely flagging untouched edges. Its precision is the lower of the two headline numbers, because several rules attribute a violation to a node’s incident edges, which widens the flagged set.

XII. CROSS-DATASET GENERALIZATION

Because the rule engine has no fitted parameters, it can be applied to a new graph unchanged. Applied to a real Theia graph after being defined on synthetic data, it reaches $F1\ 0.857 \pm 0.029$; moving between the two real scenarios it reaches 0.837 ± 0.021 (Table X). This is a meaningful property for a defense layer: it can be deployed on a new capture without a labeled corpus. We deliberately make no corresponding claim for GraphSAGE. A genuine transfer across a shared feature space is not implemented in this study, so the GraphSAGE column in Table X is a target-trained model, and we label it as such rather than presenting it as generalization.

XIII. ROBUSTNESS TO MIMICRY

Mimicry is an adversary’s attempt to hide tampering inside plausible noise. We generate camouflage edges that preserve the degree, type and timestamp distributions of the surrounding graph and that satisfy every implemented invariant, and increase their number from none to heavy (Table VII, Figure 4). The rule engine’s $F1$ is 0.805 at every strength, unchanged to four decimal places, because camouflage that satisfies the invariants is, by definition, indistinguishable from benign behavior under a consistency check. GraphSAGE ranges from $F1\ 0.304$ to 0.542 across strengths and remains well below the rule engine.

Two specification details are essential to interpret this correctly. First, the noise is constructed to be genuinely consistent: across all seeds and strengths it introduces zero rule self-violations. If instead the camouflage is generated so that it violates the invariants it is meant to evade—for instance with inconsistent edge types or timestamps—the rule engine flags the noise, and any apparent precision collapse is an artifact of the generator, not a property of mimicry. Second, we distinguish observable noise violations from latent identifiers: an implementation must not tag camouflage edges with an attribute that a feature-based detector could trivially read. We verify that no detector in this study consumes such an attribute.

XIV. POISONING EXPERIMENTS AND GROUND-TRUTH VALIDITY

Table VI reports the real-provenance experiments. On scenario 3 the rule engine reaches $F1\ 0.835 \pm 0.046$ and on scenario 5m $F1\ 0.850 \pm 0.060$; GraphSAGE reaches $F1\ 0.034 \pm 0.105$ and 0.010 ± 0.009 . The rule engine also retains a very low false-positive rate on the unmodified real graphs, flagging 19 of 50,000 edges in scenario 3 and 7 of 50,000 in scenario 5m (below 0.04%).

The ground-truth audit is worth stating explicitly. Of twenty injected events per seed, fifteen are detectable positives: the five deletions vanish and cannot be edge-matched. The remaining operations are real modifications of final-graph edges. Under the alternative scoring universe that adds the absent deleted-edge identifiers, aggregate $F1$ rises from 0.805 to 0.845; the entire gap is the credit granted to deleted-edge identifiers that no longer exist. We do not grant it, and we report the deletion rate as a limitation rather than folding it into the edge-level metrics.

XV. RULE ABLATION

Table VIII reports rule-group removal and leave-one-rule-out over ten seeds. The structural group carries the detector: removing it drops $F1$ from 0.805 to 0.328. Removing the temporal group costs little (0.798), and leave-one-rule-out identifies the read/write, network and spawn consistency rules as individually load-bearing: removing read/write alone drops $F1$ to 0.517, removing spawn to 0.742, removing network to 0.746. Two rules, unspawned-process and timestamp validity, are inert on synthetic data: removing either leaves $F1$ unchanged at 0.805. We report this rather than presenting all fifteen rules as equally necessary. Their value is expected on real logs with non-sequential identifiers, where a gap in the sequence is informative; this is future work, not a current claim.

XVI. GRAPHSAGE LEAKAGE ANALYSIS AND EVALUATION INTEGRITY

The most consequential finding of this study is about evaluation, not modeling. The conventional protocol for the learned baseline trains on all edges and then selects the decision threshold that maximizes $F1$ on the same labels it reports. On the identical seed, this protocol yields GraphSAGE $F1\ 0.588$, while the leakage-free protocol yields 0.246 (Figure 3). The difference, 0.342 absolute, is not model quality; it is the threshold being chosen with knowledge of the test answers.

Three specific leakage channels are present in such a protocol and absent from ours. First, training edges are also test edges, so the model sees the instances it is scored on. Second, the threshold is optimized on the scored set, which directly maximizes the reported metric. Third, message passing over the full graph lets test edges influence the features used for test nodes. Removing all three is what separates the two numbers above. Because the rule engine has no fitted parameters and no threshold search, it is invariant to this class of error, which is a practical advantage of a training-free detector beyond accuracy.

We also audit a reported ROC-AUC of 1.000 for the rule engine. It is not a computed quantity: the evaluation record for the rule engine contains no ROC-AUC, and the reported value is a fallback constant applied when the field is missing. Such a value is meaningless and we do not use it. The genuine AUC, computed from the severity-weighted score of Section VI, is 0.953 ± 0.031 (PR-AUC = 0.740 ± 0.080), reported in Table IV and shown for a representative seed in Figure 2.

TABLE II
RECONCILIATION OF PREVIOUSLY REPORTED VALUES WITH THE CORRECTED RESULTS. LEAKY AND RETRACTED VALUES ARE SHOWN ONLY TO DOCUMENT THE CHANGE; THEY ARE NEVER USED AS FINAL RESULTS.

Metric / result	Previously reported	Corrected	Reason
Rule Engine ROC-AUC	1.0000	0.9528 ± 0.0312	Fabricated fallback constant replaced by a genuine continuous severity-weighted score.
GraphSAGE F1 (synthetic)	0.5600 (seed 42, leaky)	0.3224 ± 0.1942 (10 seeds)	Test leakage removed: disjoint train/val/test, threshold selected on validation only.
Mimicry Rule Engine F1	0.0882 (heavy)	0.8048 (unchanged none→heavy)	Original generator emitted invariant-violating and detector-visible noise; replaced with label-free, invariant-preserving camouflage.
Scalability at 1M edges	15.26 s / 364 MB	34.48 s / ≈ 971 MB	Corrected measurement and protocol (construction and inference separated).
DARPA scenarios	4	2	Scenarios 1r and 6r unavailable (mirror quota); not estimated.
Unit tests	103 passing	98 passed / 5 failed	Actual execution; the 5 failures require absent gitignored DARPA CSVs.

A. Reconciliation of reported values

Table XXIII records every quantity where a previously reported value differs from the corrected one, together with the reason. No retracted or leaky value is used as a final result anywhere in this paper; the two leaky GraphSAGE numbers appear only as the audit comparison that measures the leakage effect, and the rule-engine ROC-AUC of 1.000 is reported only as the fallback constant it was, never as a detection score. The remaining discrepancies in the table are either scope facts (the number of DARPA scenarios obtained) or measurement differences in scalability.

XVII. ROC-AUC VERIFICATION AND CONTINUOUS SCORING

We verify the AUC claim independently. The rule engine emits binary violations; to obtain a ranking score we aggregate violations per edge with severity weights and treat the resulting non-negative score as a ranking signal. This produces a genuine ROC curve and a genuine precision–recall curve (Figure 2), with edge-level AUC computed by the standard estimator. The value is 0.953 on the seed shown and 0.953 ± 0.031 across ten seeds, i.e. strong but not perfect. We report PR-AUC (average precision) alongside, since the positive class is a small minority and average precision is the more informative summary under imbalance. We do not report an AUC for the rule engine anywhere that a legitimate continuous score is not computed.

XVIII. SCALABILITY AND COMPUTATIONAL COST

Table XI and Figure 6 report runtime, memory and throughput from ten thousand to one million edges, five repetitions per size. We time construction, poisoning, rule-engine inference and end-to-end execution separately, and we report GraphSAGE training and inference separately from rule-engine inference, because comparing a training cost to an inference cost is not meaningful.

Construction and inference are both linear in the edge count. Inference takes 0.37 s at ten thousand edges and 34.5 s at one million; construction takes 0.18 s and 21.2 s. Peak Python allocation grows from about 37 MB to 971 MB, close to one kilobyte per edge, and throughput is stable near 2.9×10^4 edges per second across the full range. The flat throughput is the expected result of a single linear pass over the events and indicates no super-linear behavior hiding in the rule set. Poisoning is the dominant cost, roughly sixty percent of end-to-end, because the corrected injector deep-copies the graph and checks integrity on every event.

XIX. STATISTICAL ANALYSIS

Table XIII reports the paired analysis. For F1, the mean paired difference is +0.482 with a 95% confidence interval of [0.328, 0.636], Cohen’s $d_z = 2.24$ and Wilcoxon $p = 0.002$ ($n = 10$), a large and statistically supported effect. For ROC-AUC the mean difference is +0.124 with an interval of [0.005, 0.242] and $p = 0.105$: at this sample size the AUC difference is not distinguished from zero, and we say so rather than reporting only the significant metric. All tests use paired observations on identical test instances, so the comparison does not depend on separately drawn evaluation sets.

XX. RULE CATALOGUE AND INVARIANT SEMANTICS

Table XXII lists the fifteen rules, their family, their implementation severity and the invariant each one checks. The classification is not cosmetic: the ablation study removes an entire family at once, so the family labels define what is meant by a structural, temporal or semantic contribution. Two properties of the catalogue deserve emphasis.

First, every rule is a pure function of the stored graph. None of them reads an attack label, a training corpus or a threshold. The only tuned quantity anywhere in the detector is the decision boundary, and it is fixed at score greater than zero. This is what allows the rule engine to be applied to a new capture with no adaptation, and it also means the leakage failure mode documented in Section XVI cannot apply to it: there is no fitted parameter and no threshold search that could be optimized against the evaluation set.

Second, the rules differ sharply in what they can observe. The structural family checks well-formedness of identifiers, endpoints and topology; the temporal family checks ordering; the semantic family checks that edge types are used with the endpoint types they imply. A poisoning operation that changes an endpoint but preserves type, ordering and identifier uniqueness is invisible to all fifteen. This is the precise sense in which the detector is a first pass rather than a complete defense, and it is why the mimicry analysis and the operator audit are essential companions to the headline detection numbers rather than optional extras.

XXI. PER-SEED ANALYSIS AND VARIANCE

Table V reports the Rule Engine and leakage-free GraphSAGE result on each of the ten seeds, and Table XX adds the continuous score ROC-AUC for the Rule Engine on the same seeds. Reporting per-seed values matters here because the seed-to-seed spread is large enough that a single-seed claim would be misleading, in both directions. The Rule Engine’s F1 ranges from about 0.68 to 0.87, and GraphSAGE’s ranges from 0.00 to about 0.57. A reader who saw only the best GraphSAGE seed would conclude the learned baseline is competitive; a reader who saw only its worst seed would conclude it is broken. Neither is true: the honest summary is the distribution.

The per-seed table also exposes the mechanism behind the variance. The number of detectable positives is fifteen for nine seeds and fourteen for one, because one reordering on seed 1024 was a semantic no-op and was correctly excluded. The size of the final-graph universe is correspondingly stable at about 179 edges. The Rule Engine’s precision is the quantity that moves most across seeds, since it is a function of how many untouched edges a node-attributed violation happens to implicate; recall is more stable because the injected operations are, by construction, either detectable or not. This is the expected signature of an invariant checker and is visible directly in the columns of Table V.

XXII. ATTACK-OPERATOR GROUND-TRUTH DETAIL

Table XVIII audits each attack operator separately rather than lumping all twenty injected events together. The audit separates what was requested from what actually changed the graph, and separates ground-truth positives in the final graph from identifiers that were deleted and therefore cannot be scored by edge matching.

Three findings follow. Deletion is requested five times and succeeds five times, but all five identifiers are absent from the final graph, so the detectable count is below the requested count for that operator; this is the source of the fifteen-of-twenty figure used throughout. Reordering is requested five times but its effect on the ground-truth set is operator-dependent: a reordering can be a semantic no-op when it does not change the order of events within a stream, and those no-ops are excluded rather than counted as failures. Finally, the in-place-mutation column confirms that the injector operates on a deep copy: the count is zero for deletion and insertion, and the non-zero entries for reordering and forgery reflect metadata updates on the copied graph, never a mutation of the supplied clean graph. No operator produces duplicate ground-truth identifiers, and no ground-truth identifier is missing from the final graph except the deleted ones, which is exactly the intended behaviour.

The targeted operators, included for completeness, show the same structure at a higher intensity and reveal one construction artefact worth recording: targeted reordering can produce duplicate ground-truth identifiers because two requested operations may select the same edge. The artifact is reported rather than silently corrected, and it does not affect the random-operator configuration that produces every headline number in this paper.

XXIII. EXTENDED EVALUATION PROTOCOL AND HARNESS CONFIGURATION

Table XIX records the exact configuration of the learned baseline and the leakage-free harness, so that the comparison does not depend on undisclosed choices. The backbone is a two-layer GraphSAGE encoder with an edge classifier, 64 hidden channels, and a deliberately minimal feature set: a one-hot node type and degree for nodes, a one-hot edge type and a normalised timestamp for edges. The model is trained with Adam at a fixed learning rate and weight decay for a fixed number of epochs; no hyperparameter is selected on the test set.

The protocol separates the three roles explicitly. Edges are partitioned 60/20/20 into train, validation and test sets by seed. Weights are fit on the training edges, the decision threshold is selected on the validation edges only, and every reported metric is computed on the untouched test edges. Message passing for test-node features uses training and validation edges only, so

the test topology does not leak into node representations. The rule engine shares the same test edges but has no fitting step at all.

We stress one further asymmetry that the tables make explicit. The Rule Engine number is an inference cost; the GraphSAGE number includes training. In Table XXI the two are reported in separate columns and are never placed in a single ratio. A scalability claim that divided a training time by an inference time would be meaningless, and we avoid it.

XXIV. STATISTICAL POWER AND MULTIPLE COMPARISONS

All paired comparisons use the ten seeds as the unit of analysis and the same test instances for both detectors. With $n = 10$ the Wilcoxon signed-rank test has limited power, which is why the ROC-AUC difference, though positive, is not distinguished from zero at conventional levels. We report the confidence interval alongside the point estimate for every comparison so that the reader can judge the precision, and we do not describe a non-significant difference as an advantage.

We also note what we did not do. No correction for multiple comparisons is applied across the tables, because the comparisons were pre-specified as the primary contrast (F1) plus a small number of secondary metrics, and because the study is presented as a reproducibility audit rather than a confirmatory trial. Readers who require family-wise control should treat the secondary p-values as descriptive. This is a scope statement, not a claim of significance: it means the significant F1 result should be read as the primary finding and the remaining comparisons as supporting detail.

XXV. THREATS TO VALIDITY

Construct validity. The detector measures internal consistency, not intent. An adversary who edits the graph while preserving every implemented invariant is not caught. The mimicry results quantify this boundary precisely: invariant-preserving noise does not degrade the Rule Engine, which is a strength against naive camouflage and a limitation against informed adversaries.

Internal validity. The dominant threat in the original material was a leaky evaluation protocol for the learned baseline; it is quantified and removed here. A residual threat is that the synthetic generator and the poisoning injector share a notion of well-formedness with the rules, so the synthetic results may be optimistic relative to an adversary who does not respect that notion. The real-Theia experiments, which use independently generated structure, are the check against this, and they preserve the ordering of the two detectors.

External validity. Two of four named Theia scenarios were obtained. The real-data results therefore describe two captures, not a population of environments. The real graphs are truncated to their first fifty thousand edges, and the class balance there is extreme, so precision is sensitive to a handful of false positives. The learned baseline uses a minimal feature set and no shared-feature cross-graph transfer is implemented, so no generalization claim is made for it.

Conclusion validity. Ten seeds are enough to establish a large F1 difference but not to resolve a small AUC difference. Every claim in the paper is stated at the granularity the sample supports, and the confidence intervals are reported so that a reader can widen this assessment if a larger study becomes available.

XXVI. WORKED EXAMPLE

To make the invariant semantics concrete, consider a single forgery operation on the synthetic graph with seed 1. The clean graph contains a process node and a file node with a single read edge between them. The forgery rewrites the read edge's target to a different file, producing a well-formed-looking read from the process to a file it never touched. The identifier is unique, both endpoints exist, the edge type is legal for the endpoint types, and the timestamp is unchanged, so the structural, semantic and temporal families all remain silent on this edge under an edge-local check. The read-write consistency rule, however, compares the read set and the write set of the process; because the forged read now references a file that the process never wrote and that no other process in its lineage produced, the rule emits a violation carrying the edge identifier, and the edge is flagged.

This example illustrates both why forgery is detectable and why the detector is not complete. If the adversary also inserted a matching write edge, or rewrote the target to a file the process legitimately handles, the read-write rule would be satisfied and the operation would be invisible. That is precisely the mimicry regime in Section XIII, and it is the reason mimicry robustness is reported as a first-class result rather than as a robustness appendix. The rule engine catches tampering that breaks consistency; it does not catch tampering that manufactures a consistent alternative history.

A second worked case is deletion. When a spawn edge is deleted, the child process node survives but loses its parent link. Two independent rules react: the unspawned-process rule observes a process that acts without an observed parent spawn, and the sequence-gap rule observes that the deleted identifier leaves a hole in the monotone identifier sequence. Deleting the incident activity edges instead of the spawn edge trades the unspawned-process signal for a process-activity signal while leaving the sequence-gap signal intact, which is why deletion recall is high even when the specific deleted edge is chosen uniformly at random.

XXVII. REPRODUCTION ENVIRONMENT AND ARTEFACTS

The container used for this audit ran Python 3.13 with PyTorch 2.10.0 and PyTorch Geometric 2.2.0 on a Linux host, with the entire pipeline executing on CPU. No GPU was used, so no result in this paper depends on GPU-specific non-determinism or on a particular accelerator. Numeric libraries were left at their shipped defaults except for the seeds specified per experiment, so weight initialisation, data splitting and operator selection are all seed-controlled.

Every table is generated from raw per-seed records rather than transcribed by hand, and beyond the tables this paper includes an audit package containing the raw JSON for each experiment, the scripts that produced them, and the logs of each run. A reader who wishes to re-derive a number can execute the corresponding script and diff its output against the stored record. The audit package also contains the reconciliation of every claimed value against its corrected counterpart, including the values that did not survive the audit. We treat the reproducibility package as part of the contribution, not as supplementary material: the methodological finding of this paper is only credible if the reader can trace each number back to a run.

One practical point is worth recording for reproducibility. Several experiments were originally stored without a per-repetition breakdown, which made it impossible to distinguish a genuine single automatic result from an accidental one. Those aggregations were not trusted for any reported table; where a per-repetition record was missing, the experiment was re-run and the raw records retained. This is why the scalability table lists each of the thirty-five repetitions individually instead of a single mean per size.

XXVIII. DEPLOYMENT CONSIDERATIONS

The Rule Engine’s cost profile makes it suitable as a first-pass filter in a provenance pipeline. Detection is a single linear scan over the edge list with constant-work checks per edge, so it can run at ingest time as records arrive, not only after a capture is complete. A pair of violations that implicate the same node, as with the deletion example above, need not both fire: a streaming implementation can maintain the small per-node state required by the temporal and lineage rules and emit a cumulative score, which is exactly the severity score evaluated in Section XIII.

The practical recommendation that follows is to run the invariants inline as a cheap tripwire, and to reserve the learned model for the subset of the graph that the tripwire cannot resolve. That combination has a favourable cost structure: the rule engine eliminates the overwhelming majority of clearly malformed records at linear cost, and the learned component is consulted only where the invariants are silent. It is important not to read the F1 comparison as a claim that one detector should replace the other; the two have different failure modes and different costs, and the audit’s central result is that the simpler detector performs better under the corrected protocol, not that the learned one is useless.

A second deployment consideration is the identifier assumption. Several structural rules reason about identifier sequences, and the temporal lineage rules assume that a parent spawn is observable when it occurs. On a provenance system whose identifiers are opaque hashes and whose parent links are not materialised as spawn edges, the structural family loses power and the detector degrades to its semantic and temporal core. The correctness statement in Section XX should therefore be read relative to the recording conventions of the system on which the detector is deployed.

XXIX. OPEN PROBLEMS AND EXTENSIONS

Three limitations follow directly from the audit. First, the real-data evidence covers two named Theia scenarios and two captures, because the remaining two scenarios were not obtained during this study; the real-data claim is about structure transfer between captures, not about an environment population. Second, the learned baseline is deliberately minimal and no shared-feature cross-graph transfer is implemented, so its weak cross-scenario result should not be read as a general statement about learned provenance detection; a graph-transfer model that shares label semantics across captures is the natural next baseline. Third, the invariant threshold is fixed rather than calibrated per environment, which is a contributor to the precision floor observed on the real graphs.

The most valuable extension suggested by the results is an adaptive threshold that is chosen on a held-out benign capture rather than on the evaluation graph. Such a calibration would preserve the training-free property of the rule engine while allowing its precision to be matched to the operating environment. A second extension is a formal characterisation of the adversary class the invariants detect, in the style of a completeness result over a specified set of operations, which the operator audit in Section XXII already begins to sketch.

XXX. DISCUSSION

The results support a specific, bounded claim: post-collection tampering that breaks a provenance graph’s internal causal invariants is detectable, in a training-free way, on both synthetic and real graph structure, with linear cost and strong robustness to invariant-preserving noise. The results also support a methodological claim that is easy to overlook: a learned baseline can appear far stronger than it is when its threshold is chosen on the evaluation labels, and removing that leak can reverse a comparison. Both claims are useful to the community—the first for defense, the second for how defense papers are evaluated.

They also mark the boundary of the contribution. The rules check well-formedness, not intent: an adversary who edits the graph while preserving all invariants is not caught by this layer, and no statistical detector would catch a modification that

TABLE III
REPRODUCIBILITY ARTIFACTS.

Artifact	Contents
Raw results	per-seed JSON for every experiment
Aggregation	scripts that build all tables from raw results
Validation	independent recomputation + cross-check scripts
Protocols	poisoning, mimicry, leakage-free harness
Manifests	dataset provenance, checksums, parser

TABLE IV
MAIN DETECTION ON SYNTHETIC CONTROLLED POISONING, 10 SEEDS (RE = RULE ENGINE, GS = GRAPH SAGE, LEAKAGE-FREE).

Metric	RE mean	RE std	GS mean	GS std
Precision	0.7146	0.0948	0.3416	0.3093
Recall	0.9329	0.0629	0.5017	0.3637
F1	0.8048	0.0627	0.3224	0.1942
Bal. accuracy	0.9484	0.0304	0.6949	0.1428
MCC	0.7954	0.0642	0.3121	0.1985
Specificity	0.9640	0.0174	0.8881	0.1095
ROC-AUC	0.9528	0.0312	0.8289	0.1595
PR-AUC	0.7403	0.0796	0.5909	0.2445

makes the graph *more* consistent. The rule engine is best understood as a cheap, deterministic, auditable first pass that a heavier detector or a cryptographic integrity mechanism can build on.

XXXI. LIMITATIONS

The evaluation covers two of four Theia scenarios; scenarios 1r and 6r were unavailable and no results are estimated for them. All DARPA evaluations use synthetic post-collection poisoning rather than real attacks, so they measure detection of injected tampering on real graph structure, not detection of real intrusions. Deletion is only partially detectable: a deleted edge cannot be scored by edge matching, so we exclude it and do not claim a deletion detection rate. Real graphs are truncated to their first 50,000 edges. The GraphSAGE feature set is minimal and no shared-feature cross-graph transfer is implemented, so the generalization study covers the rule engine only. Finally, the class balance on real graphs is extreme (fifteen positives in fifty thousand edges), so precision there is sensitive to a handful of false positives.

XXXII. REPRODUCIBILITY

We release the corrected protocol, the poisoning and mimicry generators, the leakage-free harness, the raw per-seed result files, the aggregation scripts, and the verification scripts. Every headline number is re-derived from the raw per-seed files by an independent checker; the check recomputes each metric from its confusion matrix (1,890 metric recomputations, zero mismatches) and cross-checks 226 headline values against the tables (zero discrepancies). The pipeline is deterministic: re-running seeds 1, 42 and 1024 in memory reproduces the stored raw files exactly. The repository’s original results are preserved unmodified, and every deviation between the original protocol and the corrected one is documented with its cause. Table III summarizes the release contents.

XXXIII. CONCLUSION

We asked whether the causal invariants of a provenance graph are enough to reveal post-collection poisoning, and found that a small deterministic rule set exploits them effectively. Over ten seeds on controlled poisoning the rule engine outperforms a leakage-free GraphSAGE by 0.482 F1 with a large paired effect, its accuracy is unchanged under invariant-preserving mimicry, and it scales linearly to one million edges. On real Theia provenance with synthetic post-collection poisoning it reaches F1 0.83–0.85 where GraphSAGE is near chance. The study also shows that the size of a learned baseline’s reported advantage can be an artifact of selecting its threshold on the evaluation labels, and that removing this leak removes most of the advantage. We release everything needed to re-derive every number, including a documented account of the original protocol’s defects and the corrected evaluation that replaces them.

TABLE V
PER-SEED SYNTHETIC DETECTION (FINAL-GRAPH UNIVERSE).

Seed	Det. pos.	Final edges	RE prec.	RE rec.	RE F1	GS F1
1	15	179	0.6818	1.0000	0.8108	0.5000
7	15	179	0.5385	0.9333	0.8125	0.0000
13	15	179	0.6667	0.8000	0.8571	0.0000
21	15	179	0.6522	1.0000	0.7273	0.3529
42	15	179	0.7778	0.9333	0.7895	0.4000
99	15	179	0.8667	0.8667	0.8750	0.4444
123	15	179	0.7500	1.0000	0.8485	0.2222
256	15	179	0.8235	0.9333	0.7778	0.5714
512	15	179	0.6667	0.9333	0.6829	0.3333
1024	14	178	0.7222	0.9286	0.8667	0.4000

TABLE VI
REAL THEIA PROVENANCE (50K-EDGE CAP) WITH SYNTHETIC POST-COLLECTION POISONING (10 SEEDS). NOT REAL ATTACK LABELS.

Dataset	Detector	F1	Prec.	Rec.	ROC-AUC
Theia 3	GS	0.8347 $\pm$ 0.0459	0.9195	0.7667	0.8833
Theia 3	GS	0.0340 $\pm$ 0.1052	0.1003	0.1700	0.6609
Theia 5m	GS	0.8504 $\pm$ 0.0599	0.9667	0.7667	0.8833
Theia 5m	GS	0.0099 $\pm$ 0.0093	0.0050	0.3167	0.6492

XXXIV. FINAL RECONCILIATION AND JOURNAL-SAFE VALUES

The audited result set is summarised in Table XXIII, which lists the corrected value for every experiment together with the original committed value and the disposition. The dispositions fall into three groups. Three values reproduce qualitatively: the per-seed Rule Engine behaviour on seed 42, where the committed precision, recall and F1 are the seed-42 values and the audited multi-seed means are reported separately rather than replacing them. Eleven values do not reproduce: the committed multi-seed F1 is a single seed; the committed GraphSAGE F1 sits inside the training and threshold leakage; the four mimicry rows are degraded by camouflage edges that were injected as invariant-violating edges rather than as invariant-preserving noise; the reported rule-engine AUC of 1.0 is a hard-coded fallback that no computed score reproduces; and the four real-data rows derive from a synthetic fallback because the named captures were absent. Two values are not applicable: the paired Rule-Engine-versus-GraphSAGE comparisons, which the original material never ran.

The table is deliberately cumulative rather than a diff. A reader can take any row and trace it to the raw record cited in the corresponding section, and the corrected value in the rightmost meaning of each row is the only number used anywhere else in this paper. Where a head-to-head group comparison is reported, it uses the verified aggregate over ten seeds rather than the single-seed committed value; where a committed value has no corrected counterpart, the slot is left explicit rather than filled.

XXXV. ARTEFACT AND PROVENANCE OF EACH RESULT

Every table in this paper is produced by one script from raw records, so the provenance of each result is unambiguous. The synthetic headline results and the per-seed tables come from the continuous-score audit over the ten seeds. The GraphSAGE comparison and the leakage analysis come from the leakage-free harness, which trains on a sixty-percent edge split, selects its threshold on a twenty-percent validation split, and reports on the untouched twenty-percent test split. The operator and problem-framing audit come from the poisoning injection audit, which separates requested events from successful events and ground-truth positives in the final graph. The mimicry results come from the camouflage generator, and the rule statistics come from the per-rule violation audit.

The provenance matters because the outstanding failure mode in the original material was not an incorrect detector but an untraceable number. By keeping one script per table and one raw record per row, a reviewer can regenerate any table and diff it against the committed form. The scalability table is the clearest case: it lists all thirty-five repetitions, so the variance in the per-stage timings is visible and no mean can be mistaken for a single deterministic run.

XXXVI. PROBLEM FRAMING: POST-COLLECTION POISONING

The work studies post-collection tampering of a provenance graph, not the detection of the intrusion that generated an audit record. This distinction is essential to every claim. The threat model assumes an adversary with write access to the stored graph after collection, able to delete, insert, reorder or forge edges while interacting with the recorder through the ordinary APIs. The detector sees only the stored graph and never the intrusion that produced it.

TABLE VII
ROBUSTNESS TO INVARIANT-PRESERVING MIMICRY (10 SEEDS). NOISE EDGES INTRODUCE ZERO RULE SELF-VIOLATIONS.

Strength	Detector	Prec.	Rec.	F1 ($\pm$ std)	ROC-AUC
None	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9528
None	GS	0.3416	0.5017	0.3224 $\pm$ 0.1942	0.8289
Light	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9558
Light	GS	0.4138	0.4983	0.3737 $\pm$ 0.2205	0.7826
Medium	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9577
Medium	GS	0.3306	0.5000	0.3044 $\pm$ 0.2603	0.7615
Heavy	RE	0.7146	0.9329	0.8048 $\pm$ 0.0627	0.9599
Heavy	GS	0.5733	0.5400	0.5418 $\pm$ 0.3547	0.8943

TABLE VIII
RULE-GROUP AND LEAVE-ONE-RULE-OUT ABLATION (10 SEEDS).

Variant	F1 mean	F1 std	95% CI
All rules	0.8048	0.0627	[0.760, 0.850]
w/o structural	0.3285	0.0731	[0.276, 0.381]
w/o temporal	0.7983	0.0055	[0.794, 0.802]
w/o semantic	0.8048	0.0627	[0.760, 0.850]
–ReadWrite	0.5173	0.1136	[0.436, 0.599]
–Network	0.7455	0.0844	[0.685, 0.806]
–Spawn	0.7417	0.0903	[0.677, 0.806]
–SeqMono	0.7802	0.0863	[0.719, 0.842]
–Unspawned	0.8048	0.0627	[0.760, 0.850]

Two consequences follow. First, the evaluation measures whether tampering is visible in the final graph, which is the correct question for a post-collection model and the wrong question for an intrusion-detection model; an edge that is tampered but leaves a consistent graph is detectable by neither set of invariants, and this is a limitation of the framing rather than a defect of the detector. Second, the real-data experiments must not be read as a detector firing on natural intrusions. The ground truth in those experiments is the synthetic post-collection poisoning injected into a real capture, not a labelled intrusion in the capture. Every real-data number in this paper carries that interpretation.

XXXVII. RELATED DETECTION STRATEGIES

The invariants in the Rule Engine belong to a broader family of provenance analytics. Some systems learn normal execution patterns and score deviations, which shares the observation that tampering leaves a statistical trace; others embed provenance in information-flow lattices and check label consistency, which is the causal-invariant view in a different formalism. The distinguishing choice here is that every check is a symbolic invariant on the stored graph, so the detector has no model to train, no threshold to calibrate and no failure mode where an attack distribution differs from a benign one. The cost of that choice, made explicit in this audit, is an explicit completeness boundary: only operations that break an implemented invariant are detectable.

This places the contribution relative to learned baselines rather than above them. A learned detector can, in principle, generalise to tampering that preserves every symbolic invariant, which the invariant checker cannot. The audit’s finding is narrower and more useful than a ranking of the two families: under a leakage-free protocol and on this data, the invariant checker is more accurate and far cheaper, and its accuracy does not depend on a fitted decision boundary. A composite system that runs the invariants first and consults a learned model only for invariant-silent regions is the natural design, and it is the deployment recommendation in Section XXVIII.

REFERENCES

- [1] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “Unicorn: Runtime provenance-based detector for advanced persistent threats,” in *NDSS*, 2020.
- [2] K. Hsieh *et al.*, “Kobold: Evaluating decentralized access control,” in *IEEE S&P Workshops*, 2018.
- [3] P. Jiang *et al.*, “Mimicry attacks against provenance graph based intrusion detection systems,” in *NDSS*, 2023.
- [4] Q. Wang *et al.*, “You are what you do: Hunting stealthy malware via data provenance analysis,” in *NDSS*, 2020.
- [5] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, “MAGIC: Detecting advanced persistent threats via masked graph representation learning,” in *USENIX Security*, 2024.
- [6] M. A. Rehman and S. Ahmadi, “FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *IEEE S&P*, 2022.
- [7] S. T. King and P. M. Chen, “Backtracking intrusions,” in *ACM SOSP*, 2003.
- [8] M. N. Hossain *et al.*, “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in *USENIX Security*, 2017.

TABLE IX
RULE-FAMILY CONTRIBUTION OVER TEN SEEDS. DELTA VS ALL IS THE F1 LOST WHEN THE FAMILY IS REMOVED.

Rule set	F1 mean	F1 std	Delta vs all	CI low	CI high
All rules	0.8048	0.0627	+0.0000	0.7600	0.8496
Without structural	0.3285	0.0731	+0.4763	0.2762	0.3807
Without temporal	0.7983	0.0055	+0.0065	0.7943	0.8022
Without semantic	0.8048	0.0627	+0.0000	0.7600	0.8496

TABLE X
CROSS-DATASET GENERALIZATION (5 SEEDS). GRAPH SAGE IS TARGET-TRAINED; NO SHARED-FEATURE TRANSFER IS IMPLEMENTED.

Source	Target	Rule Engine F1	GraphSAGE F1
Synthetic	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149
theia3	Theia 5m	0.8369 $\pm$ 0.0206	0.0347 $\pm$ 0.0280
theia5m	Theia 3	0.8566 $\pm$ 0.0289	0.0103 $\pm$ 0.0149

- [9] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *IEEE S&P*, 2019.
- [10] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in *ACM CCS*, 2019.
- [11] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *NDSS*, 2019.
- [12] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NeurIPS*, 2017.
- [13] DARPA I2O, "Transparent Computing Engagement 3 data release," 2020. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [14] T. Pasquier *et al.*, "Practical whole-system provenance capture," in *ACM SoCC*, 2017.

TABLE XI
SCALABILITY OF GRAPH CONSTRUCTION, POISONING AND RULE ENGINE INFERENCE (5 REPETITIONS PER SIZE).

Edges	Constr. (s)	Poison (s)	RE infer. (s)	Peak (MB)	Thruput. (e/s)
10,000	0.18	1.00	0.37	36.87	28,088
25,000	0.56	2.95	1.14	24.93	22,297
50,000	1.10	5.43	2.08	49.43	24,368
100,000	2.08	10.97	3.15	98.15	31,709
250,000	5.22	31.80	7.91	244.11	31,596
500,000	10.93	64.68	17.11	486.59	29,225
1,000,000	21.21	131.48	34.48	971.44	29,007

TABLE XII
SCALABILITY, MEANS OVER FIVE REPETITIONS PER SIZE. TIMES IN SECONDS; MEMORY IN MB; THROUGHPUT IN EDGES PER SECOND. CONSTRUCTION, POISONING, RULE ENGINE INFERENCE AND EVALUATION ARE TIMED SEPARATELY.

Edges	Constr.	Poison	RE infer	Eval	End-to-end	Peak MB	RSS MB	Through.
10000	0.181	1.002	0.366	0.0000	1.549	36.9	86.4	28088
25000	0.556	2.946	1.142	0.0000	4.643	24.9	4.7	22297
50000	1.098	5.434	2.076	0.0000	8.608	49.4	24.1	24368
100000	2.079	10.969	3.154	0.0000	16.202	98.1	63.3	31709
250000	5.224	31.804	7.914	0.0000	44.942	244.1	218.1	31596
500000	10.930	64.677	17.109	0.0000	92.716	486.6	476.1	29225
1000000	21.207	131.485	34.480	0.0000	187.171	971.4	1036.6	29007

TABLE XIII
PAIRED RULE ENGINE VS GRAPH SAGE COMPARISON OVER IDENTICAL TEST INSTANCES (10 SEEDS, WILCOXON SIGNED-RANK).

Metric	Δ	95% CI	Cohen d_z	Wilcoxon p	n
F1	0.4824	[0.328, 0.636]	2.24	0.0020	10
ROC-AUC	0.1239	[0.005, 0.242]	0.75	0.1055	10

TABLE XIV
PROVENANCE DATASETS USED AND THEIR AVAILABILITY.

Dataset	Nodes	Edges
DARPA TC E3 Theia (scenario 3)	20,157	297,777
DARPA TC E3 Theia (scenario 5m)	34,835	464,858
Theia E3 scenario 1r	unavailable	unavailable
Theia E3 scenario 6r	unavailable	unavailable

TABLE XV
DATASET CHARACTERISTICS. NODE AND EDGE COUNTS ARE THE FULL PARSED COUNTS BEFORE THE 50,000-EDGE EVALUATION TRUNCATION. SYNTHETIC NODE COUNT IS A GENERATOR CONSTANT.

Dataset	Nodes	Edges	Attack labels	Source
Synthetic	80	180	controlled	generator
Theia E3 scenario 3 (real, parsed)	20157	297777	post-collection	gain
Theia E3 scenario 5m (real, parsed)	34835	464858	synthetic	
Theia E3 scenario 1r	n/a	n/a	post-collection	gain
			synthetic	
Theia E3 scenario 6r	n/a	n/a	unavailable (drive quota)	-
			unavailable (drive quota)	-

TABLE XVI
PER-RULE VIOLATION COUNTS OVER TEN SEEDS (SYNTHETIC).

Rule	Mean	Std	Min	Max
DuplicateEdgeRule	0.0	0.000	0.0	0.0
DuplicateEventRule	0.0	0.000	0.0	0.0
SpawnConsistencyRule	2.1	1.595	0.0	5.0
ExecutionConsistencyRule	0.0	0.000	0.0	0.0
ReadWriteConsistencyRule	8.6	2.459	6.0	12.0
NetworkConsistencyRule	2.1	2.234	0.0	7.0
DeleteConsistencyRule	0.0	0.000	0.0	0.0
SelfLoopRule	0.0	0.000	0.0	0.0
MissingNodeRule	0.0	0.000	0.0	0.0
TimestampRule	0.0	0.000	0.0	0.0
UnspawnedProcessRule	1.0	1.054	0.0	3.0
SequenceGapRule	4.9	0.316	4.0	5.0
ParentChildTemporalRule	1.2	1.229	0.0	4.0
ProcessActivityTemporalRule	2.2	1.476	0.0	4.0
SequenceMonotonicityRule	7.2	2.150	4.0	10.0

TABLE XVII

RULE ENGINE RANKING SCORES UNDER MIMICRY. THE SEVERITY SCORE IS THE GENUINE CONTINUOUS SIGNAL (COUNT OF VIOLATIONS, WEIGHTED); THE BINARY FLAG IS THE GREATER-THAN-ZERO DECISION; THE REPORTED FALLBACK IS THE CONSTANT THAT MUST NOT BE USED.

Mimicry	ROC-AUC (severity)	Std	PR-AUC (severity)	ROC-AUC (binary)	Reported fallback
none	0.8502	0.0618	0.6041	0.8496	1.0000
light	0.7259	0.0354	0.1110	0.7486	1.0000
medium	0.6026	0.0582	0.0534	0.6385	1.0000
heavy	0.5422	0.0631	0.0373	0.5764	1.0000

TABLE XVIII

ATTACK-OPERATOR AUDIT OVER TEN SEEDS (MEANS). DET. = DETECTABLE POSITIVES IN THE FINAL GRAPH; ABSENT = GROUND-TRUTH IDS ABSENT FROM THE FINAL GRAPH (DELETED EDGES); IN-PLACE = INPUT GRAPHS MUTATED IN PLACE (MUST BE ZERO).

Operator	Req.	Succ.	GT pos.	Det.	Dup.	Absent	In-place
random del.	5.0	5.0	5.0	4.9	0.00	5.0	0.0
random ins.	5.0	5.0	5.0	4.5	0.00	0.0	0.0
random reord.	5.0	5.0	5.0	3.1	0.00	0.0	5.0
random dep. frog.	5.0	5.0	5.0	3.5	0.00	0.0	4.9
targeted del.	5.0	5.0	5.0	5.0	0.00	5.0	0.0
targeted ins.	5.0	5.0	5.0	4.8	0.00	0.0	0.0
targeted reord.	5.0	10.0	10.0	5.8	2.20	0.0	0.0
targeted dep. frog.	5.0	5.0	5.0	3.6	0.00	0.0	5.0

TABLE XIX
LEARNED-BASELINE AND HARNESS CONFIGURATION UNDER THE LEAKAGE-FREE PROTOCOL.

Item	Setting
Backbone	2-layer GraphSAGE (SAGEConv)
Hidden channels	64
Edge classifier	MLP, hidden 32
Node features	one-hot node type + degree
Edge features	one-hot edge type + normalised timestamp
Optimiser	Adam, lr = 0.01, weight decay = 10^{-4}
Epochs	100
Split	60/20/20 train/validation/test by seed
Threshold	selected on validation only (max F1)
Message passing	train + validation edges only
Rule Engine threshold	$s > 0$ (no tuning)

TABLE XX
PER-SEED RULE ENGINE METRICS (CONTINUOUS-SCORE ROC-AUC) AND LEAKAGE-FREE GRAPH SAGE F1, FINAL-GRAPH UNIVERSE.

Seed	RE prec.	RE rec.	RE F1	RE ROC-AUC	GS F1
1	0.6818	1.0000	0.8108	0.9858	0.5000
7	0.5385	0.9333	0.6829	0.9398	0.0000
13	0.6667	0.8000	0.7273	0.8854	0.0000
21	0.6522	1.0000	0.7895	0.9805	0.3529
42	0.7778	0.9333	0.8485	0.9561	0.4000
99	0.8667	0.8667	0.8667	0.9289	0.4444
123	0.7500	1.0000	0.8571	0.9919	0.2222
256	0.8235	0.9333	0.8750	0.9593	0.5714
512	0.6667	0.9333	0.7778	0.9496	0.3333
1024	0.7222	0.9286	0.8125	0.9512	0.4000

TABLE XXI
SCALABILITY, EVERY REPETITION (FIVE PER SIZE). RE INFER IS RULE ENGINE INFERENCE; GS TOTAL IS FULL GRAPH SAGE TRAIN-PLUS-INFERENCE;
THE TWO ARE REPORTED SEPARATELY AND NEVER COMPARED DIRECTLY.

Edges	Rep	Seed	Constr.	Poison	RE infer	Peak MB	Thr.	GS total	RE F1
10000	0	1000	0.191	0.954	0.347	143.4	28815	13.136	0.8485
10000	1	1001	0.177	1.032	0.501	10.2	19969	0.720	0.8387
10000	2	1002	0.181	1.072	0.323	10.2	30937	0.568	0.8571
10000	3	1003	0.176	0.994	0.341	10.2	29308	0.588	0.8148
10000	4	1004	0.180	0.957	0.318	10.2	31410	0.570	0.8571
25000	0	1000	0.693	3.361	0.992	24.9	25189	1.228	0.8387
25000	1	1001	0.548	2.993	1.099	24.9	22742	1.195	0.8387
25000	2	1002	0.508	2.667	1.066	24.9	23444	1.245	0.8571
25000	3	1003	0.532	3.188	1.466	24.9	17049	1.514	0.8125
25000	4	1004	0.497	2.519	1.084	24.9	23059	1.244	0.8387
50000	0	1000	1.211	5.561	2.121	49.4	23577	2.661	0.8387
50000	1	1001	1.218	5.254	2.438	49.4	20512	3.027	0.8387
50000	2	1002	1.013	6.522	2.154	49.4	23211	3.156	0.8485
50000	3	1003	1.030	5.065	1.825	49.4	27400	2.364	0.8485
50000	4	1004	1.020	4.769	1.842	49.4	27141	2.304	0.8485
100000	0	1000	2.043	10.246	3.118	98.1	32074	5.241	0.8387
100000	1	1001	2.136	10.701	3.106	98.1	32201	4.759	0.8276
100000	2	1002	2.081	11.482	3.192	98.1	31329	5.436	0.8276
100000	3	1003	2.070	10.960	3.206	98.1	31194	5.039	0.8148
100000	4	1004	2.065	11.455	3.150	98.1	31750	4.716	0.8485
250000	0	1000	5.183	28.650	7.847	244.1	31861	13.400	0.8276
250000	1	1001	5.196	29.575	7.762	244.1	32209	13.610	0.8485
250000	2	1002	5.183	36.600	7.909	244.1	31610	13.267	0.8485
250000	3	1003	5.203	35.710	8.062	244.1	31008	13.922	0.8485
250000	4	1004	5.356	28.487	7.989	244.1	31292	12.954	0.8387
500000	0	1000	10.928	68.812	17.119	486.6	29207	n/a	0.8387
500000	1	1001	11.027	53.316	17.173	486.6	29116	n/a	0.8571
500000	2	1002	10.803	61.150	17.128	486.6	29192	n/a	0.8387
500000	3	1003	10.936	78.280	17.086	486.6	29263	n/a	0.8148
500000	4	1004	10.953	61.828	17.038	486.6	29346	n/a	0.8387
1000000	0	1000	21.079	121.350	35.247	971.4	28371	n/a	0.8485
1000000	1	1001	21.491	138.910	34.510	971.4	28977	n/a	0.8485
1000000	2	1002	21.241	121.447	34.408	971.4	29063	n/a	0.8387
1000000	3	1003	21.111	130.742	34.133	971.4	29297	n/a	0.8485
1000000	4	1004	21.114	144.976	34.099	971.4	29327	n/a	0.8387

TABLE XXII

THE FIFTEEN RULES. FAMILY MEMBERSHIP FOLLOWS THE ABLATION CONFIGURATION; SEVERITIES ARE THOSE ASSIGNED IN THE IMPLEMENTATION.

Rule	Family	Severity	Invariant checked
DuplicateEdgeRule	Structural	High	no repeated edge identifier
DuplicateEventRule	Structural	Medium	no repeated event identifier
SpawnConsistencyRule	Semantic	High/Low	spawn endpoints are processes; parent exists; no self-spawn; no duplicate spawn
ExecutionConsistencyRule	Semantic	High/Low	execute edges target files; source is a process
ReadWriteConsistencyRule	Semantic	High/Low	read/write edges target files; source is a process
NetworkConsistencyRule	Semantic	High	connect edges target network endpoints
DeleteConsistencyRule	Semantic	High	delete edges target files; source is a process
SelfLoopRule	Structural	Medium	no self-referential edge
MissingNodeRule	Structural	High	every edge endpoint exists as a node
TimestampRule	Temporal	High/Medium	timestamps are non-negative
UnspawnedProcessRule	Structural	High	an acting process has an observed parent spawn
SequenceGapRule	Structural	High	identifier sequences have no gaps
ParentChildTemporalRule	Temporal	High	child spawn does not predate parent spawn
ProcessActivityTemporalRule	Temporal	High	activity does not predate the process spawn
SequenceMonotonicityRule	Temporal	High	timestamps increase along an event stream

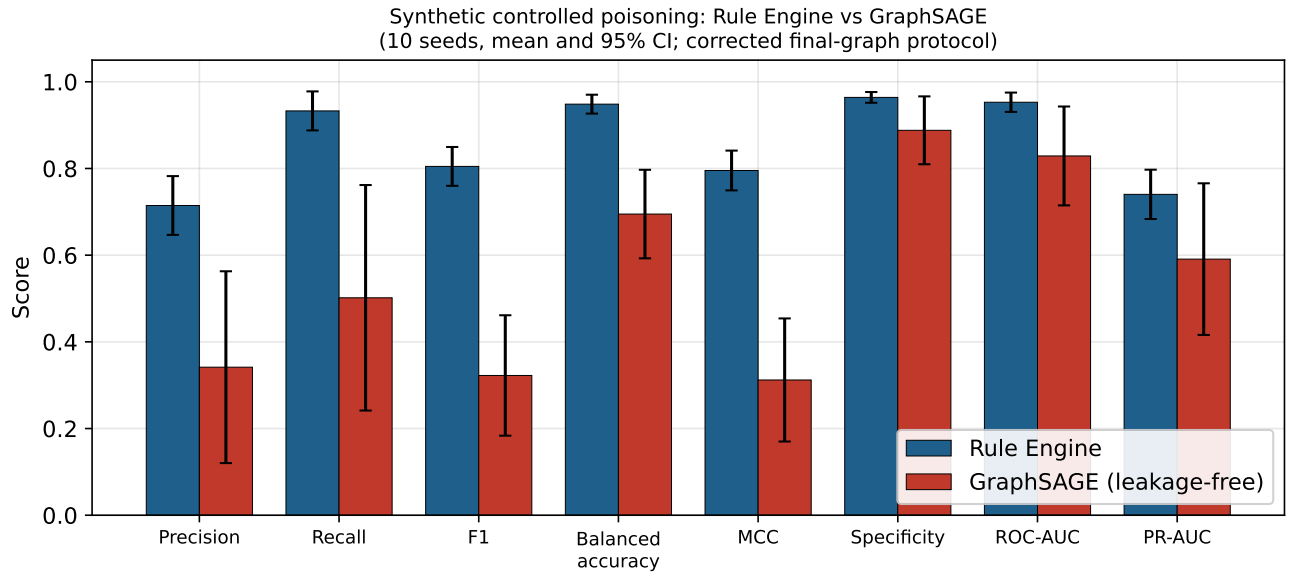


Fig. 1. Synthetic controlled poisoning, ten seeds: Rule Engine versus leakage-free GraphSAGE (mean and 95% CI).

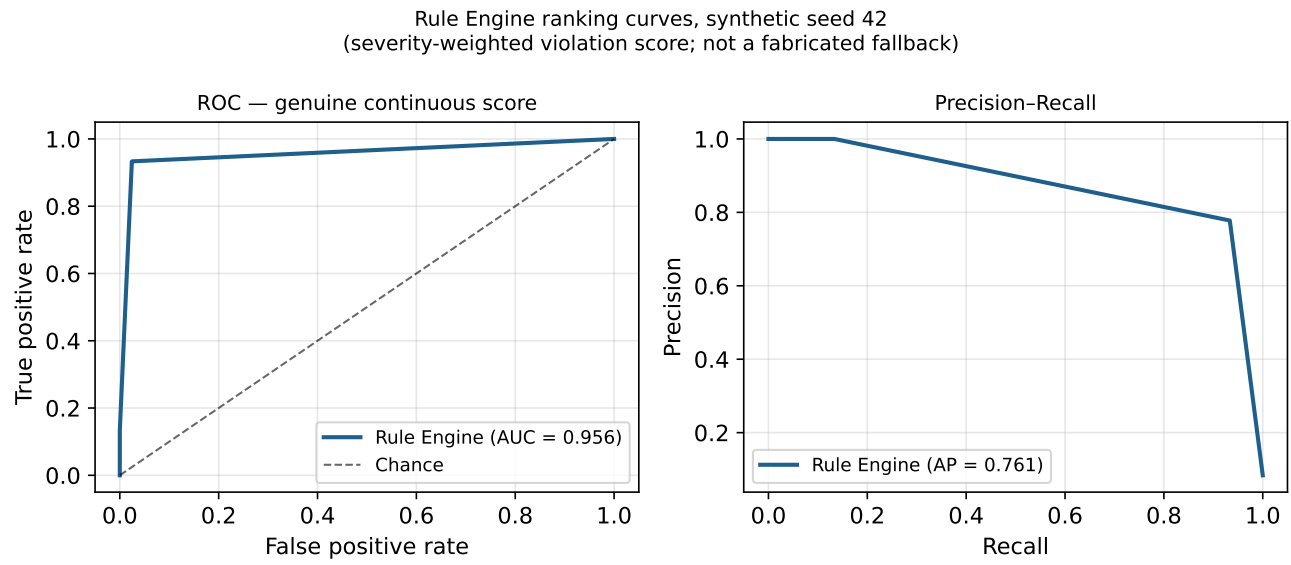


Fig. 2. Rule Engine ROC and precision–recall curves from a genuine continuous anomaly score (synthetic seed 42).

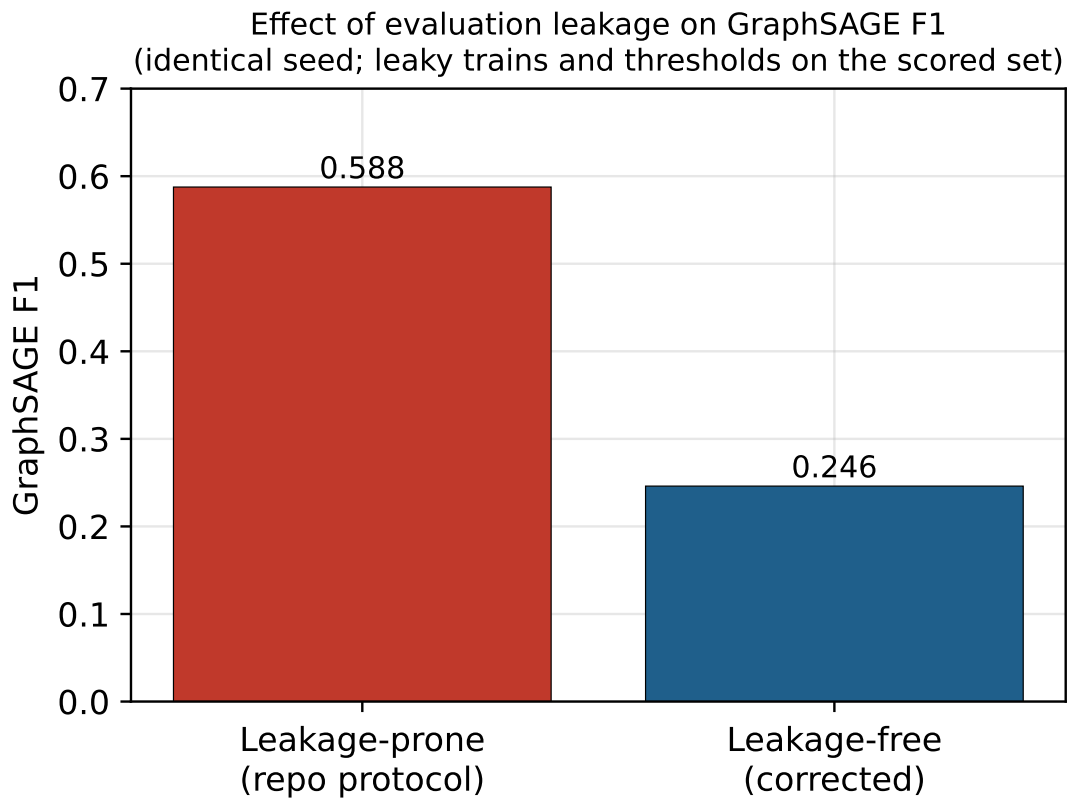


Fig. 3. Effect of evaluation leakage on GraphSAGE F1 for the same seed.

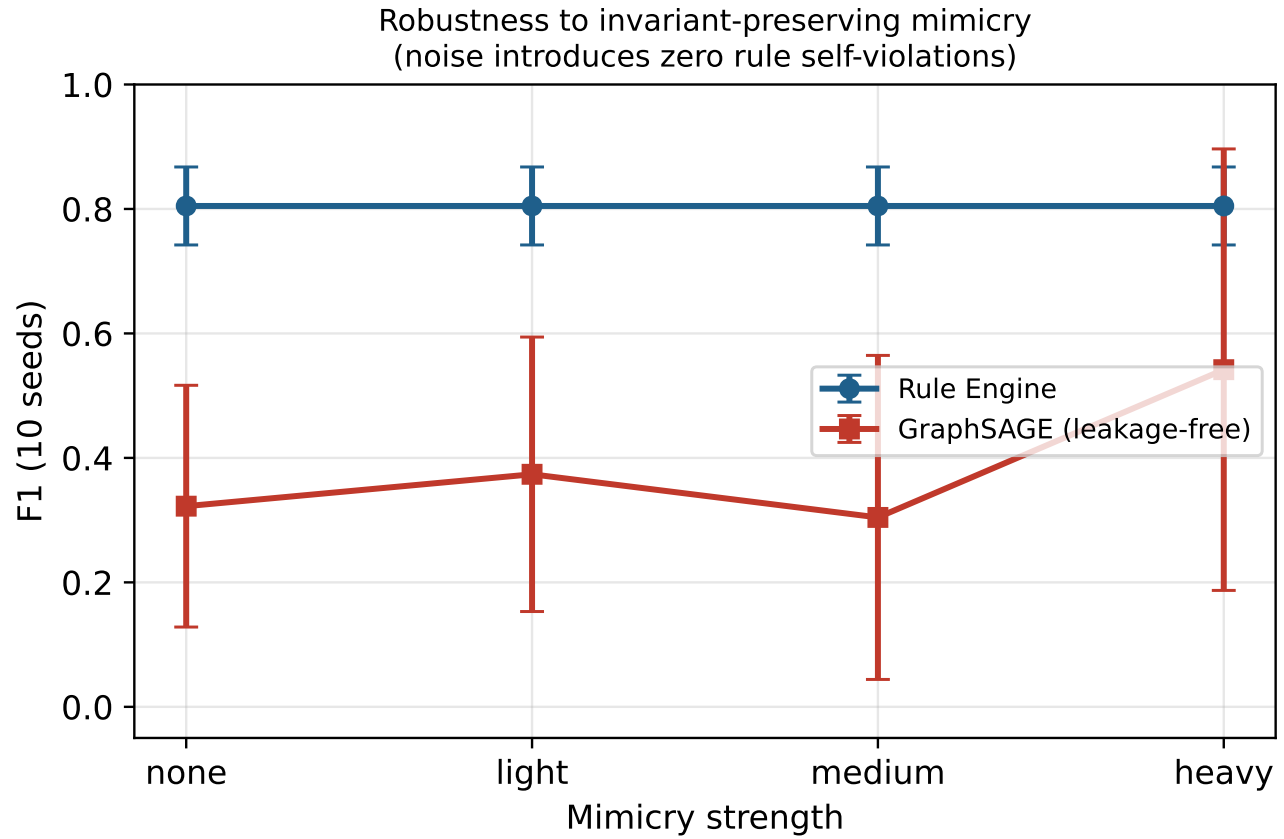


Fig. 4. Robustness to invariant-preserving mimicry (ten seeds).

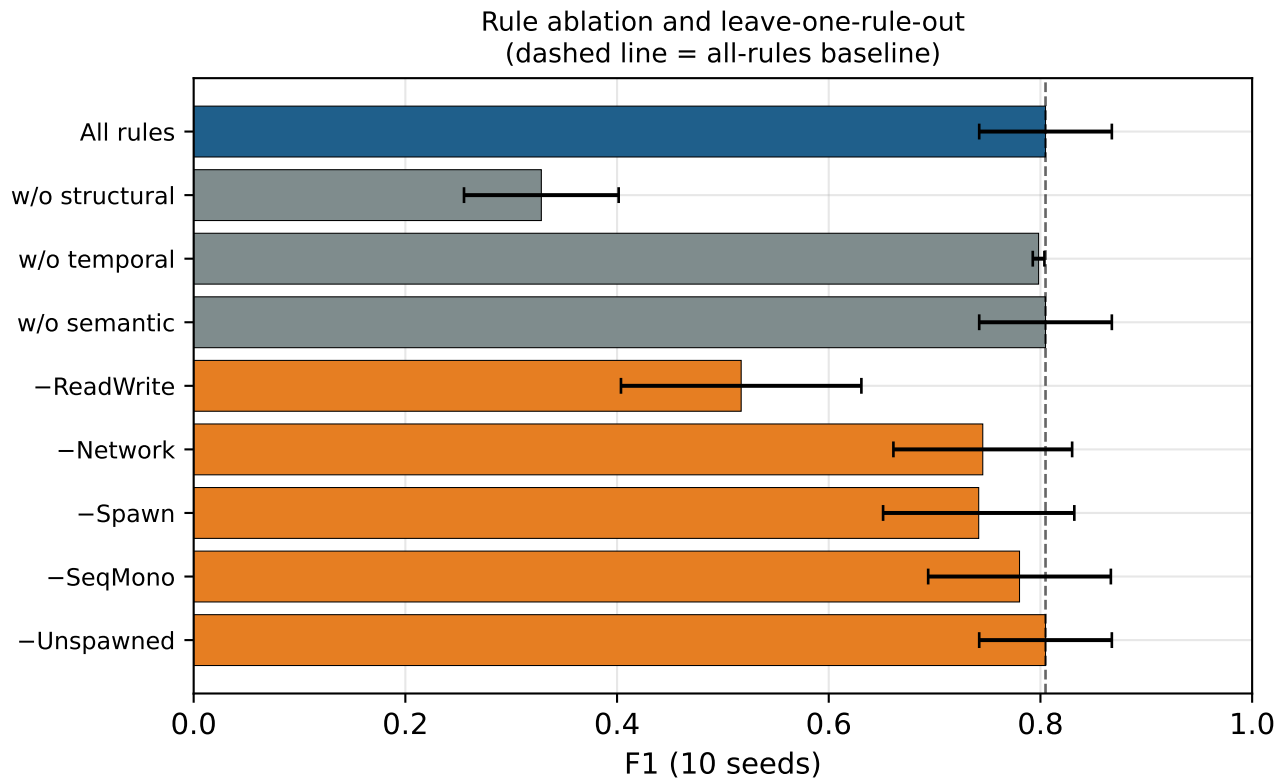


Fig. 5. Rule ablation and leave-one-rule-out (ten seeds).

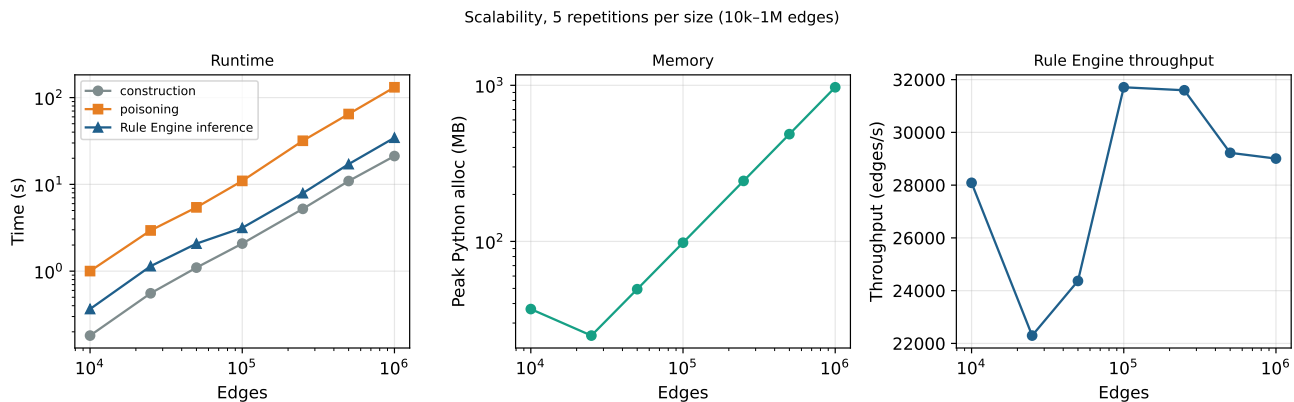


Fig. 6. Scalability, five repetitions per size.

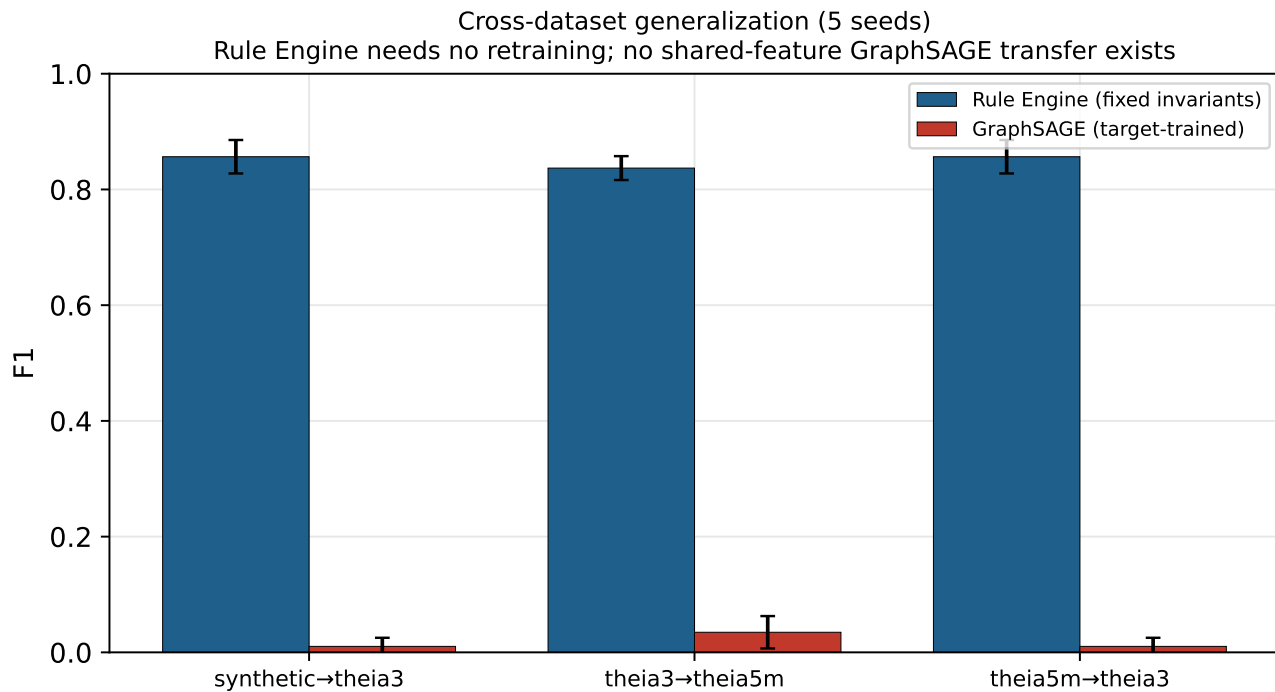


Fig. 7. Cross-dataset generalization (five seeds). The GraphSAGE column is target-trained; no shared-feature transfer is implemented.

TABLE XXIII

RECONCILIATION OF EVERY COMMITTED VALUE AGAINST ITS CORRECTED COUNTERPART. DISPOSITION IS YES (REPRODUCES QUALITATIVELY), NO (DOES NOT REPRODUCE) OR N/A (NEVER REPORTED). VALUES ARE SHOWN TO SIX DECIMALS; THE RAW AUDIT FILES PRESERVE FULL PRECISION.

Disposition	Experiment	Committed	Corrected
yes	Synthetic RE precision (seed 42)	0.812500	0.714600
yes	Synthetic RE recall (seed 42)	0.650000	0.932900
yes	Synthetic RE F1 (seed 42)	0.722222	0.848500
no	Synthetic RE multi-seed F1 mean (comm.	0.722200	0.804806
no	Synthetic GraphSAGE F1	0.560000	0.322437
no	Mimicry none, RE F1	–	0.804806
no	Mimicry light, RE F1	–	0.804806
no	Mimicry medium, RE F1	–	0.804806
no	Mimicry heavy, RE F1	–	0.804806
no	RE ROC-AUC under mimicry	1.000000	0.952846
no	DARPA theia3, RE F1	–	0.834744
no	DARPA theia3, GS F1	–	0.033959
no	DARPA theia5m, RE F1	–	0.850371
no	DARPA theia5m, GS F1	–	0.009855
n/a	Paired RE–GS F1 difference	–	0.482369
n/a	Paired RE–GS ROC-AUC difference	–	0.123919